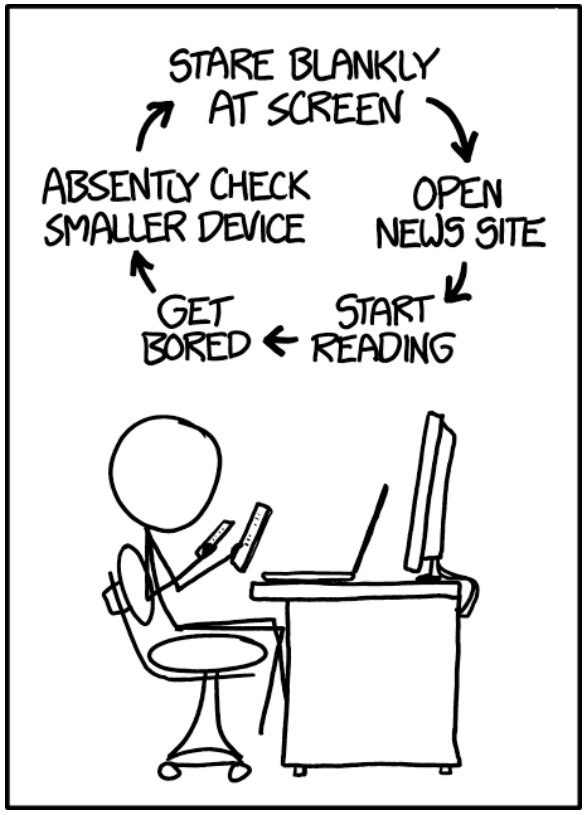
Reference and Practice Handout

Lecture-03: Branching, Procedures, and Loops

Computer Organization & Assembly Language Programming
CS 2160 || Fall 2026
Instructor: Himon Thakur



Loop
(Randall Munroe; CC BY-NC 2.5; https://xkcd.com/1411/)

This document is a supplementary resource alongside course materials and class lectures. You are expected to go through and utilize it by yourself. You may read a section, then try solving the short exercises before moving on. Answers are provided at the end.

This handout will cover:

- How **branch instructions** let a program make decisions.
- Turning **if/else** statements into assembly, using a "*goto*" version as a stepping stone.
- Turning **while**, **do-while**, and **for** loops into assembly with reusable *templates*.
- Calling **procedures** (functions), returning from them, and using the **stack** to save registers.

I used Claude Opus 5.5 to help me format the LaTeX code. All contents were produced, edited, or verified by me.

1. Instructions for Making Decisions

Normally the processor runs instructions one after another. A **branch** compares two registers and, if the test is true, jumps to a **label** instead. Every `if`, `while`, and `for` in C is built from this one idea.

A **label** (like `Exit:`) is just a name for the address of the instruction after it. It takes no space in memory.

Instruction	Jumps to `L` if...	Comparison
`beq rs1, rs2, L`	`rs1` $=$ `rs2`	
`bne rs1, rs2, L`	`rs1` $\neq$ `rs2`	
`blt rs1, rs2, L`	`rs1` $<$ `rs2`	signed
`bge rs1, rs2, L`	`rs1` $\geq$ `rs2`	signed
`bltu rs1, rs2, L`	`rs1` $<$ `rs2`	unsigned
`bgeu rs1, rs2, L`	`rs1` $\geq$ `rs2`	unsigned

- There is no "branch if greater than" instruction. Swap the operands instead: $a > b$ is the same as $b < a$, so use `blt b, a, L`. (Assemblers accept `bgt`/`ble` as shorthand for exactly this.)
- **Unconditional jump** ("goto"): `j L`, which the assembler turns into `jal x0, L`. You will also see `beq x0, x0, L`: since $0 = 0$ is always true, it always jumps.

Exercise 1

Which single instruction jumps to `L` when `a0` $>$ `a1` (signed)? Which jumps when `a0` $\leq$ `a1`?

2. If / Else

Assembly has no `{ }` blocks. The trick: rewrite the C code using only `if (...) goto` and plain `goto`. Each of those lines becomes one instruction.

2.1. Worked example

```
if (i == j) { f = g + h; }
else        { f = g - h; }
```

Register map: `t0` $= f$, `t1` $= g$, `t2` $= h$, `t3` $= i$, `t4` $= j$.

There are two equally correct ways to write the goto version: branch on the condition being *true*, or on it being *false*.

Version 1: jump if true

```
if (i == j) goto Then;
f = g - h;        // else part
goto Exit;
Then: f = g + h;
Exit:
```

```
      beq  t3, t4, Then
      sub  t0, t1, t2
      j    Exit
Then: add  t0, t1, t2
Exit:
```

Version 2: jump if false

```
if (i != j) goto Else;
f = g + h;        // then part
goto Exit;
Else: f = g - h;
Exit:
```

```
      bne  t3, t4, Else
      add  t0, t1, t2
      j    Exit
Else: sub  t0, t1, t2
Exit:
```

Watch out

Don't forget the `j Exit` after the first block. Without it, the processor "falls through" into the second block and runs *both* parts.

2.2. The general template

C	Template 1	Template 2 (usually shorter)

`if (cond) {`
`  body_if`
`} else {`
`  body_else`
`}`
	`if (cond) goto If;`
	`  body_else`
	`  goto Exit;`
	`If:  body_if`
	`Exit:`
		`if (!cond) goto Else;`
		`  body_if`
		`  goto Exit;`
		`Else:  body_else`
		`Exit:`

With no `else`, template 2 is just: `if (!cond) goto Exit; body_if; Exit:`.

Example 1: `abs` with an if/else

```
int abs(int v) {
    int r;
    if (v < 0) r = -v;
    else       r =  v;
    return r;
}
```

Goto version: `cond = v < 0; if (cond) goto Then; goto Done; Then:  r = -v; Done:  return r;`

```
abs:  slt  t0, a0, x0     # t0 = 1 if v < 0, else 0
      bne  t0, x0, Then   # v < 0?  go negate
      j    Exit           # otherwise v is already the answer (r = v)
Then: sub  a0, x0, a0     # a0 = -v
Exit: ret                 # return a0
```

Here `r` lives in `a0`, the same register that holds `v` and the return value, so "`r = v`" needs no instruction.

Exercise 2

Translate `if (a < b) a = b;` (so that `a0` ends up holding $\max(a, b)$), with `a0` $= a$ and `a1` $= b$. Use template 2 (one branch, one other instruction).

Exercise 3

Translate `if (x == 0) y = 1; else y = 2;` with `a0` $= x$, `a1` $= y$.

3. While Loops

A loop is an if-statement whose last step is "`goto` back to the top". The only design choice is *where* to put the test.

3.1. Worked example

```
while (A[i] == k) { i += 1; }
```

Register map: `a0` $= i$, `a1` $= k$, `a2` = `&A[0]` (address of the array). `A` is an `int` array, so `A[i]` lives at address `&A[0]` $+4i$.

Template #1: test at the top

```
Loop:
  if (!Cond) goto Exit;
  Body
  goto Loop;
Exit:
```

Assembly

```
Loop: slli t0, a0, 2       # t0 = i * 4
      add  t0, t0, a2      # t0 = &A[i]
      lw   t1, 0(t0)       # t1 = A[i]
      bne  t1, a1, Exit    # A[i] != k ? leave
      addi a0, a0, 1       # i = i + 1
      j    Loop            # go to Loop
Exit:
```

3.2. Template #2: "jump to the middle"

Put the test at the *bottom*, and jump straight to it once at the start. Each pass through the loop now costs **one** branch instead of two.

Template #2

```
  goto Test;
Loop:
  Body
Test:
  if (Cond) goto Loop;
Exit:
```

Assembly

```
      j    Test            # jump to the test first
Loop: addi a0, a0, 1       # i = i + 1
Test: slli t0, a0, 2       # t0 = i * 4
      add  t0, t0, a2      # t0 = &A[i]
      lw   t1, 0(t0)       # t1 = A[i]
      beq  t1, a1, Loop    # A[i] == k ? repeat
Exit:
```

Notice the branch flipped from `bne ...Exit` ("leave if false") to `beq ...Loop` ("repeat if true").

Exercise 4

Suppose `A[0]`, `A[1]`, `A[2]` equal `k` and `A[3]` does not, with $i = 0$ at the start. How many branch or jump instructions run in total with template #1? With template #2?

4. Example: Counting 1-Bits (`popcount`)

Count how many bits of `x` are 1: look at the lowest bit (`x & 1`), add it to the total, shift `x` right, and repeat until `x` is 0.

C (while)

```
int pcount_while(unsigned int x) {
  int result = 0;
  while (x) {
    result += x & 1;
    x >>= 1;
  }
  return result;
}
```

Goto version (template #2)

```
int pcount_goto_jtm(unsigned int x) {
  int result = 0;
  goto Test;
Loop:
  result += x & 1;
  x >>= 1;
Test:
  if (x) goto Loop;
  return result;
}
```

Assembly

```
      addi t0, x0, 0       # result = 0
      j    Test            # goto Test
Loop: andi t1, a0, 1       # t1 = x & 1
      add  t0, t0, t1      # result += t1
      srli a0, a0, 1       # x >>= 1   (logical: x is unsigned)
Test: bne  a0, x0, Loop    # if (x != 0) goto Loop
      addi a0, t0, 0       # return value = result
      ret
```

Example 2: Trace with $x = 11 = 1011_2$

Pass	`x & 1`	result after	x after `>>= 1`
1	1	1	`101` (5)
2	1	2	`10` (2)
3	0	2	`1` (1)
4	1	3	`0` (0) $\rightarrow$ stop

Returns 3.

Exercise 5

A classmate wrote the loop body as `andi t1, x0, 1`. What does the function return for every input, and why?

5. Do-While Loops

A `do-while` runs the body **first** and tests afterwards, so the body always runs *at least once*. Its goto version needs no initial jump at all.

C

```
do
  Body
while (Cond);
```

Goto version

```
Loop:
  Body
  if (Cond) goto Loop;
```

popcount (do-while)

```
int result = 0;
do {
  result += x & 1;
  x >>= 1;
} while (x);
```

```
      addi t0, zero, 0      # result = 0
Loop: andi t1, a0, 1        # t1 = x & 1
      add  t0, t0, t1       # result += t1
      srli a0, a0, 1        # x >>= 1
      bne  a0, x0, Loop     # if (x) goto Loop
      addi a0, t0, 0        # return result
      ret
```

Compare with Section 4: template #2 for `while` is exactly the do-while loop plus one "`j Test`" at the start, which skips the body when the condition is false from the beginning.

Exercise 6

For $x = 0$, how many times does the loop body run in the `while` version? In the `do-while` version? Do the two return the same answer here?

6. For Loops

A `for` loop is a `while` loop with the setup and the update written on one line. Convert it to a `while`, then use a while template.

for version

```
for (Init; Cond; Update)
    Body
```

while version

```
Init;
while (Cond) {
    Body
    Update;
}
```

Example 3: popcount with a `for` loop

```
#define WSIZE 8*sizeof(int)       // 32 bits
int pcount_for(unsigned int x) {
    size_t i; int r;
    for (r = 0, i = 0; i < WSIZE; i++)
        r += (x >> i) & 1;
    return r;
}
```

Init: `r = 0, i = 0` **Cond:** `i < WSIZE` **Update:** `i++` **Body:** `r += (x » i) & 1`
Using template #1 (`a0` $= x$, `t0` $= r$, `t1` $= i$):

```
      addi t0, x0, 0       # r = 0
      addi t1, x0, 0       # i = 0
      addi t2, x0, 32      # t2 = WSIZE
Loop: bgeu t1, t2, Exit    # if !(i < WSIZE) goto Exit   (i is unsigned size_t)
      srl  t3, a0, t1      # t3 = x >> i
```

```
      andi t3, t3, 1        # t3 = (x >> i) & 1
      add  t0, t0, t3       # r += t3
      addi t1, t1, 1        # i++
      j    Loop
Exit: addi a0, t0, 0        # return r
      ret
```

Unlike the `while` version, this always makes 32 passes, even when `x` is 0.

Exercise 7

Translate this into RISC-V. `a0` = `&A[0]` (an `int` array), `a1` = n; return the sum in `a0`.

```
int sum = 0;
for (int i = 0; i < n; i++) sum += A[i];
```

7. Procedures and the Stack

Every function shares the same 32 registers. When one function calls another, both want to use `ra`, `a0`, `s0`, and so on. The **stack** is scratch memory where a function parks values it must not lose.

7.1. Calling and returning

- **Call:** `jal ra, func` jumps to `func` and saves the address of the *next* instruction in `ra` (the *return address*).
- **Return:** `jalr x0, 0(ra)` (shorthand: `ret`) jumps back to the address in `ra`.
- Arguments go in `a0`–`a7`; the result comes back in `a0` (and `a1`).

Kind of function	Example	Needs the stack?
Leaf (calls nothing)	`abs`	Usually not. Use `t` and `a` registers freely.
Non-leaf (calls others)	`absadd`, `main`	Yes. Its own `jal` overwrites `ra`, so `ra` must be saved.

Recall the register rules: `s0`–`s11` and `sp` must be returned unchanged (callee-saved); `t0`–`t6` and `a0`–`a7` may be overwritten by any call (caller-saved).

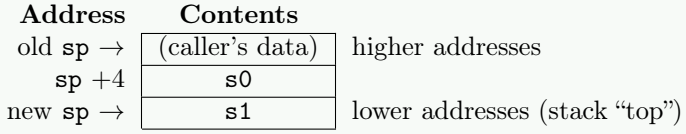
7.2. How the RISC-V stack works

- The stack is a block of memory that **grows down**, toward lower addresses.
- `sp` (stack pointer) holds the address of the current **top** item, which is the *lowest* address in use.
- **Push** = move `sp` down to make room, then store. **Pop** = load (if you need the value), then move `sp` back up.

Example 4: Push two registers, then pop them

```
# push
addi sp, sp, -8     # room for 2 words
sw   s0, 4(sp)
sw   s1, 0(sp)
```

```
# pop (reverse order)
lw   s1, 0(sp)
lw   s0, 4(sp)
addi sp, sp, 8      # free 2 words
```



Watch out

Every `addi sp, sp, -N` must be matched by an `addi sp, sp, N` before the function returns. If not, the caller's `sp` is wrong and its own saved values are read from the wrong place.

Exercise 8

Before a function runs `addi sp, sp, -12`, `sp` = `0x7FFFFFF0`. At what addresses do `8(sp)`, `4(sp)`, and `0(sp)` point afterwards?

8. Putting It Together: `caller` → `absadd` → `abs`

```c
int caller() { return absadd(-1, 2); }
int absadd(int x, int y) { int t0 = abs(x); int t1 = abs(y); return t0 + t1; }
int abs(int v) { int mask = v >> 31; return (v + mask) ^ mask; }
```

```
caller:
  400500: addi sp, sp, -4      # make room
  400504: sw   ra, 0(sp)       # save caller's own return address
  400508: addi a0, x0, -1      # arg0 = -1
  40050C: addi a1, x0, 2       # arg1 = 2
  400510: jal  ra, absadd      # call absadd(-1, 2); ra = 0x400514
  400514: lw   ra, 0(sp)       # restore ra
  400518: addi sp, sp, 4       # free the stack space
  40051C: jalr x0, 0(ra)       # return (a0 = 3)

absadd:
  400540: addi sp, sp, -12     # allocate 3 words
  400544: sw   ra, 8(sp)       # save return address (the jal below will overwrite ra)
  400548: sw   s0, 4(sp)       # save s0 (callee-saved: we are about to use it)
  40054C: sw   a1, 0(sp)       # save y (abs is allowed to overwrite a1)
  400550: jal  ra, abs         # abs(x); ra = 0x400554
  400554: add  s0, a0, x0      # s0 = abs(x)   (s0 survives the next call)
  400558: lw   a0, 0(sp)       # a0 = y
  40055C: jal  ra, abs         # abs(y); ra = 0x400560
  400560: add  a0, a0, s0      # a0 = |x| + |y|
  400564: lw   s0, 4(sp)       # restore s0
  400568: lw   ra, 8(sp)       # restore return address
  40056C: addi sp, sp, 12      # free the stack
  400570: jalr x0, 0(ra)       # return to caller

abs:                           # leaf: no stack needed
  400574: srai t0, a0, 31      # mask = v >> 31
  400578: add  a0, a0, t0      # v + mask
  40057C: xor  a0, a0, t0      # (v + mask) ^ mask
  400580: jalr x0, 0(ra)       # return
```

Example 5: What the stack holds while `abs` runs (start: `sp` = `0x120`)

Address	Contents	Saved by
`0x11C`	caller's return address	`caller` (at 400504)
`0x118`	`0x00400514`	`absadd` (its `ra`)
`0x114`	caller's old `s0`	`absadd`
`0x110`	$y = 2$	`absadd` ← `sp`

Example 6: A closer look: `pc`, `sp`, `ra` at key moments

Moment	Instruction	pc	sp	ra
caller calls absadd	`jal ra, absadd`	0x400510	0x11C	→ 0x400514
absadd allocates	`addi sp, sp, -12`	0x400540	→ 0x110	0x400514
absadd saves `ra`	`sw ra, 8(sp)`	0x400544	0x110	0x400514
second call to abs	`jal ra, abs`	0x40055C	0x110	→ 0x400560
absadd restores `ra`	`lw ra, 8(sp)`	0x400568	0x110	→ 0x400514
absadd frees stack	`addi sp, sp, 12`	0x40056C	→ 0x11C	0x400514
absadd returns	`jalr x0, 0(ra)`	0x400570	0x11C	0x400514

Arrows (→) mark the value *after* that instruction. The final `jalr` sends `pc` back to `0x400514` in `caller`.

Exercise 9

Why does `absadd` save `a1` (`y`) on the stack, but not `a0` (`x`)? And why does `abs` not touch the stack at all?

Recap

Remember these

1. Branches: `beq`, `bne`, `blt`, `bge` (+ `bltu`, `bgeu` for unsigned). For $>$ or $\leq$, swap the operands.
2. Unconditional jump: `j L` = `jal x0, L`.
3. To translate C: first rewrite it using only `if (...) goto` and `goto`; then each line becomes an instruction.
4. If/else: branch on the *opposite* condition to the else-part, and end the then-part with `j Exit`.
5. While: test at the top (2 branches per pass) or jump to a test at the bottom (1 branch per pass).
6. Do-while: body first, test at the bottom; it always runs at least once.
7. For = Init; while (Cond) { Body; Update; }.
8. Call with `jal ra, f`; return with `jalr x0, 0(ra)` (`ret`).
9. The stack grows down. Push: `addi sp, sp, -N` then `sw`. Pop: `lw` then `addi sp, sp, N`. Always balance them.
10. Non-leaf functions must save `ra`; any function that uses `s` registers must save and restore them.

Answer key for Exercises

Ex. 1. $a0 > a1$: `blt a1, a0, L`. $a0 \leq a1$: `bge a1, a0, L`.

Ex. 2.

```
      bge  a0, a1, Done    # if !(a < b) skip
      addi a0, a1, 0       # a = b
Done:
```

Ex. 3.

```
      bne  a0, x0, Else    # if (x != 0) goto Else
      addi a1, x0, 1       # y = 1
      j    Exit
Else: addi a1, x0, 2       # y = 2
Exit:
```

Ex. 4. Template #1: 4 `bne` (3 not taken, 1 taken) + 3 `j` = **7**. Template #2: 1 `j` + 4 `beq` (3 taken, 1 not) = **5**.

Ex. 5. It always returns **0**. `x0` is always zero, so `x0 & 1` $= 0$ and `result` never grows. The loop still ends, because `a0` is still shifted. The correct line is `andi t1, a0, 1`.

Ex. 6. `while`: 0 times (the first test fails). `do-while`: 1 time (adds $0 \,\&\, 1 = 0$, then `x` stays 0 and the loop ends). Both return 0, but only because the body happens to do nothing harmful when $x = 0$.

Ex. 7.

```
      addi t0, x0, 0       # sum = 0
      addi t1, x0, 0       # i = 0
Loop: bge  t1, a1, Exit    # if !(i < n) goto Exit
      slli t2, t1, 2       # t2 = i * 4
      add  t2, t2, a0      # t2 = &A[i]
      lw   t3, 0(t2)       # t3 = A[i]
      add  t0, t0, t3      # sum += A[i]
      addi t1, t1, 1       # i++
      j    Loop
Exit: addi a0, t0, 0       # return sum
```

(Checked: `A` $= \{3, -1, 10, 4\}$, $n = 4$ gives 16; $n = 0$ gives 0.)

Ex. 8. New `sp` = `0x7FFFFFE4`. `8(sp)` = `0x7FFFFFEC`, `4(sp)` = `0x7FFFFFE8`, `0(sp)` = `0x7FFFFFE4`.

Ex. 9. `absadd` still needs `y` *after* the first call to `abs`, and `a1` is caller-saved, so `abs` could overwrite it. `x` is used up by the first call, so there is nothing to keep. `abs` is a leaf: it calls nothing (so `ra` is never overwritten) and uses only `t0` and `a0`, which it is allowed to change.

Student Practice Worksheet

Part 1: Branches & Conditions

Q1. Write one branch instruction for each (signed values in `a0`, `a1`):

- (a) goto `L` if `a0` > 0: _____
- (b) goto `L` if `a0` $\leq$ `a1`: _____
- (c) goto `L` if `a0` $<$ `a1`, treating both as *unsigned*: _____

Q2. In an if/else translation, what goes wrong if you leave out the `j Exit` after the first block?

Part 2: Loops

Q3. Translate using the jump-to-middle template (`a0` $= x$):

```
while (x > 0) { x = x - 3; }
```

What is `a0` at the end if x starts at 10? _____

Q4. Translate into RISC-V (`t0` = `sum`, `t1` = `i`), then give the final value of `sum`:

```
int sum = 0;
for (int i = 10; i > 0; i--) sum += i;
```

Q5. Rewrite `pcount_for` from Section 6 as a `while` loop in C.

Part 3: Procedures & the Stack

Q6. A non-leaf function uses `s1` and `s2`. Write its prologue (save) and epilogue (restore and return).

Q7. Fill in the stack right after `absadd`'s first four instructions (start: `sp` = `0x120` before `caller` runs).

Address	Contents
0x118	
0x114	
0x110	

Value of `sp`: `0x`_____

Q8. Suppose `absadd` forgot to save and restore `ra`. Where would its final `jalr x0, 0(ra)` jump to, and what would happen?

Step-by-Step Solutions & Explanations

Q1: Branches

- (a) $a0 > 0$ is $0 < a0$: `blt x0, a0, L`.
- (b) $a0 \leq a1$ is $a1 \geq a0$: `bge a1, a0, L`.
- (c) `bltu a0, a1, L`.

Q2: Missing `j Exit` Answer: After the first block finishes, the processor keeps going to the next instruction, which is the first instruction of the other block. Both blocks run, so the result is whatever the second block computes.

Q3: `while (x > 0) x = x - 3;`

```
      j    Test            # jump to the test first
Loop: addi a0, a0, -3      # x = x - 3
Test: blt  x0, a0, Loop    # if (0 < x) goto Loop
```

$10 \to 7 \to 4 \to 1 \to -2$, and $-2 > 0$ is false, so `a0` $= \mathbf{-2}$.

Q4: Counting down

```
      addi t0, x0, 0       # sum = 0
      addi t1, x0, 10      # i = 10
Loop: bge  x0, t1, Exit    # if !(i > 0), i.e. 0 >= i, goto Exit
      add  t0, t0, t1      # sum += i
      addi t1, t1, -1      # i--
      j    Loop
Exit:
```

`sum` $= 10 + 9 + \cdots + 1 = \mathbf{55}$.

Q5: `pcount_for` as a while loop

```
int pcount_for_while(unsigned int x) {
    size_t i; int r;
    r = 0; i = 0;                // Init
    while (i < WSIZE) {          // Cond
        r += (x >> i) & 1;       // Body
        i++;                     // Update
    }
    return r;
}
```

Q6: Prologue and epilogue

```
func: addi sp, sp, -12     # 3 words: ra, s1, s2
      sw   ra, 8(sp)
      sw   s1, 4(sp)
      sw   s2, 0(sp)
      # ... body (may call other functions) ...
      lw   s2, 0(sp)
      lw   s1, 4(sp)
      lw   ra, 8(sp)
      addi sp, sp, 12
      jalr x0, 0(ra)       # ret
```

Q7: Stack contents

- `caller` moved `sp` to `0x11C`; `absadd` moved it to `0x110`.
- `0x118`: `0x00400514` (return address into `caller`). `0x114`: the old `s0`. `0x110`: $y = 2$.
- `sp` $=$ `0x110`.

Q8: Forgetting to save `ra` Answer: The second `jal ra, abs` set `ra` $=$ `0x400560`. Without restoring it, `jalr x0, 0(ra)` jumps to `0x400560`, inside `absadd` itself, instead of back to `caller` at `0x400514`. The function re-runs its last few instructions and never returns to its caller correctly: it keeps jumping back to `0x400560` in an endless loop.